

Code2Edge

Deployment with proof

A Bob-native workflow that traces an ML repository's inference pipeline, generates a constrained-device implementation, and proves mathematical correctness through stage-wise differential parity — before and after real hardware deployment.

IBM Bob 2.0 Hackathon (lablab.ai)

48-hour build

Target hardware: STM32U585 (ARM Cortex-M33) on Arduino UNO Q
Reference workload: Keyword spotting, DS-CNN, 119,372 parameters
Repository: <https://github.com/IndependentSalt69/Code2Edge>

This document was authored by Person C (Bob workflow, MCP server, integration and submission) by reading the full repository history, contracts, source, and evidence artifacts as of the commit noted in Section 4.

Contents

1	Basic Information	2
1.1	Project Title	2
1.2	Short Description	2
1.3	Long Description	2
1.3.1	The problem	2
1.3.2	The approach	2
1.3.3	Why a human checkpoint	3
1.3.4	Reference workload	3
1.3.5	What actually got proven, honestly	3
1.4	IBM Bob Usage Statement	3
1.5	Technology & Category Tags	4
2	System Architecture	5
2.1	Component map	5
3	Verification Methodology	7
3.1	Two-tier differential parity	7
3.2	Stage tolerance matrix (design specification)	7
3.3	Repair loop	7
4	Results	8
4.1	Edge readiness (real)	8
4.2	Host differential parity (real)	8
4.3	Device parity and physical benchmarking	8
4.4	The real deployment run	9
4.5	Engineering hygiene	9
5	Team & Attribution	10
5.1	Roles	10
5.2	Third-party attribution	10

1 Basic Information

1.1 Project Title

Code2Edge

1.2 Short Description

Elevator pitch

Code2Edge gives IBM Bob hardware awareness through an MCP server, so it can trace an ML repository’s audio pipeline, generate an equivalent C implementation for a microcontroller, and *prove* the two are numerically equivalent — stage by stage, on real hardware — instead of just hoping the ported code works.

1.3 Long Description

1.3.1 The problem

Deploying a PyTorch model to a microcontroller is not primarily a model problem — it is a *pipeline* problem. A keyword-spotting model looks simple: resample audio, window it, take an FFT, apply a Mel filterbank, take a log, normalize, and feed the result to a small CNN. In practice this chain is scattered across several files, with constants (window length, hop size, FFT convention, epsilon values, per-channel normalization statistics) buried in configuration that nobody reads twice. Reimplementing that chain in C/C++ for an MCU is exactly where deployments silently break: a wrong window size, an FFT power-vs-magnitude convention mismatch, or a log epsilon that differs by one order of magnitude will still *run* on the device — it will simply predict differently, and nothing in a black-box “does it boot” test will tell you why.

Most edge-deployment tooling treats the on-device model as exactly that: a black box, checked only by comparing final top-1 predictions. When accuracy degrades after deployment, engineers have no visibility into *which* stage of the pipeline diverged.

1.3.2 The approach

Code2Edge attacks this with two ideas that reinforce each other:

1. **Stage-wise differential parity, not black-box testing.** The same audio corpus is pushed through the Python/PyTorch reference pipeline and the generated C pipeline, and every intermediate tensor is compared: `max_abs_diff`, `mean_abs_diff`, cosine similarity, and (once the network runs) end-to-end prediction agreement and accuracy delta. When a stage’s divergence exceeds its tolerance, the system reports the *first* divergent stage and a deterministic diagnosis hint (e.g. “shape mismatch at framing → check hop length”), so debugging starts from evidence, not guesswork.
2. **Bob as an orchestrator with hardware awareness, not a black-box code generator.** An MCP server exposes five hardware-aware tools — `profile_model`, `inspect_pipeline`, `check_target`, `run_parity_test`, `benchmark_target` — plus a small set of workflow tools (`start_run`, `gate_step`, `record_approval`). Bob calls these tools to reason about the repository, the target’s memory budget, and the parity results, and drives a fixed workflow: analysis → Edge Readiness Report → *human approval checkpoint* → code generation → host parity gate (repair loop, capped at 3 attempts, then escalate to a human) → MCU build → device parity gate (same repair/escalate logic) → benchmark → predicted-vs-measured report → git branch and pull request.

1.3.3 Why a human checkpoint

Code2Edge deliberately puts a human in the loop *before* any code generation: Bob presents an Edge Readiness Report (does the model fit the target’s memory budget, what pipeline constants were found, what risks exist) and waits for an explicit APPROVE or REJECT. This is enforced in code (`workflow/approval.py`), not just in the workflow’s documentation — generation cannot start without a recorded approval, and a later approval overrides an earlier rejection for the same run.

1.3.4 Reference workload

Code2Edge is grounded in a real, trained model rather than a toy example: a Depthwise-Separable CNN (DS-CNN, 119,372 parameters) trained for 12-class keyword spotting on the Google Speech Commands v0.02 dataset, reaching 96.65% test accuracy ([tiny-kws](#) by Priyadeep Jaiswal, vendored under `reference/tiny-kws/`, MIT licensed, pinned to a specific upstream commit with SHA-256-verified file integrity). The target is the STM32U585 MCU subsystem (ARM Cortex-M33, 160 MHz, 786 KB SRAM, 2 MB Flash) on the Arduino UNO Q, whose Linux-side Dragonwing QRB2210 MPU runs the Python reference and orchestration side.

1.3.5 What actually got proven, honestly

The team holds itself to the same standard the tool enforces on the pipeline: every number in this submission is labeled as either *real* (measured or computed from a genuine artifact) or *mock* (a placeholder used when a hardware or data dependency was unavailable), and the two are never blended silently. Section 4 gives the full breakdown; in short, host-side differential parity is **real and fully passing** (500 samples, 4 pipeline stages, 2500/2500 stage evaluations), and physical benchmarking is now **real and passing** on the actual STM32U585 silicon (50/50 iterations correct). The one piece still mocked in the latest end-to-end deployment run is device parity’s corpus-wide coverage — a real single-fixture, single-stage on-device comparison exists and passes, but not yet a full 500-sample sweep — with a hardware-validation guide checked into the deploy branch for completing it.

1.4 IBM Bob Usage Statement

How Bob was used

Code2Edge is built *for* Bob, not merely *with* Bob: the MCP server is the hardware-awareness layer that lets Bob reason about a model repository and a memory-constrained target the way a human embedded engineer would, and `workflow/WORKFLOW.md` is a literal, step-by-step execution spec written for Bob to follow, including the exact tool calls, the human-checkpoint protocol, and the retry/escalation policy for both parity gates.

Concretely:

- **MCP server as Bob’s hardware senses.** Ten tools (`profile_model`, `inspect_pipeline`, `check_target`, `run_parity_test`, `benchmark_target`, `quantize_model`, plus `start_run`, `get_run_status`, `gate_step`, `record_approval`) are registered in Bob’s project MCP configuration (`.bob/mcp.json`) and were exercised directly through Bob during initial development (schema-validated mock responses, then real adapters as Person A and Person B’s outputs became available).
- **Bob drove early repository and MCP scaffolding.** The project’s initial local setup, repository survey, workspace scaffold, JSON-Schema tool contracts, and the first mock MCP server implementation were built in interactive Bob sessions; those sessions were exported and

are checked into `bob_sessions/` as part of this submission, alongside session transcripts for every later phase of the work.

- **The orchestration logic was authored generically for Bob to execute.** The parity-gate retry/escalation logic (`workflow/gates.py`) is written once and shared by both the host and device gates, enforcing a hard cap of 3 repair attempts before writing a structured escalation report for human debugging — this cap is enforced in code, not left to the calling agent’s discretion, so it behaves identically regardless of which agent (Bob, or a Bob-compatible execution environment) is driving it.
- **Transparency on execution environment.** During this 48-hour build the team could not confirm that the specific Bob build available supported defining a new project-level custom mode (only the three built-in Agent/Ask/Plan modes were discoverable in `.bob/`), so later phases of the same Bob-authored workflow were executed through an equivalent MCP-driven agent session following `workflow/WORKFLOW.md` verbatim, including its documented Fallback prompt for exactly this situation. Every one of those sessions is exported to `bob_sessions/` with the same rigor as the native Bob exports, so the full decision trail — tool calls, gate decisions, human approvals — is auditable end to end.

1.5 Technology & Category Tags

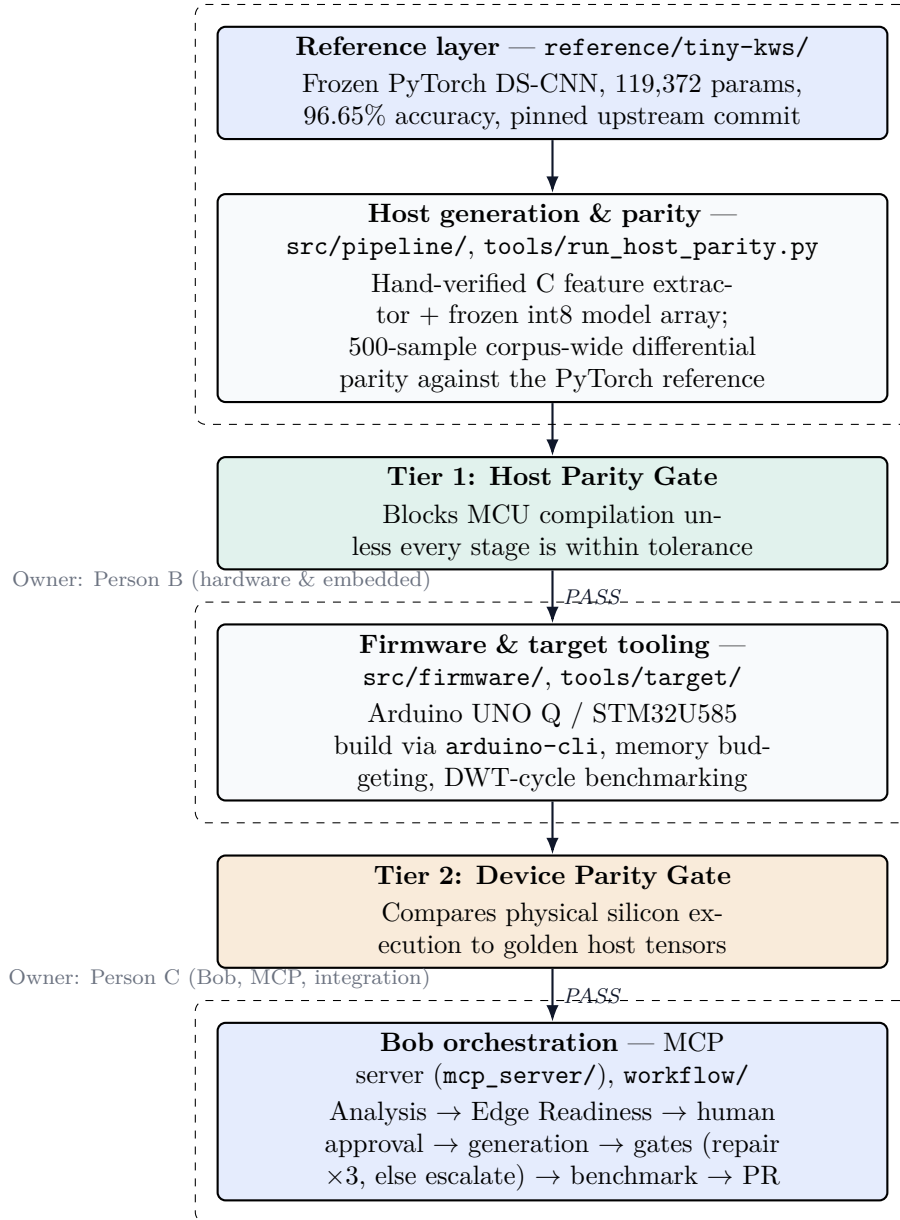
Categories: Agentic AI / Developer Tools • Edge AI & TinyML • Embedded Systems • MLOps & Verification • Hardware-in-the-loop Testing

Category	Technologies
Agent orchestration	IBM Bob (MCP client), Model Context Protocol (<code>mcp</code> Python SDK, FastMCP, stdio transport)
Backend / tooling	Python 3.10+, <code>jsonschema</code> + <code>referencing</code> (Draft 2020-12), <code>pytest</code> + <code>pytest-asyncio</code> , <code>pyserial</code>
ML / reference model	PyTorch, <code>torchaudio</code> (<code>MelSpectrogram</code>), Depthwise-Separable CNN (DS-CNN), Google Speech Commands v0.02
Edge code generation	Hand-verified C99 (Hann-windowed STFT, HTK Mel filterbank, log-compression, normalization), frozen int8 model array, TensorFlow Lite Micro / CMSIS-NN operator set
Embedded target	STM32U585 (ARM Cortex-M33, TrustZone, FPU, Armv8-M DSP), Arduino UNO Q (Qualcomm Dragonwing QRB2210 host MPU), <code>arduino-cli</code> (<code>arduino:zephyr:unoq</code>), Zephyr RTOS
Verification	Stage-wise differential parity (max/mean absolute error, cosine similarity), DWT-cycle-counter benchmarking, GCC linker-map memory accounting
Version control & CI hygiene	Git branch-per-owner workflow, Conventional Commits, JSON-Schema-gated contracts between team members, GitHub PRs merged as merge commits

2 System Architecture

Code2Edge is organized into three tiers, mirroring who owns what and how data flows from a frozen research artifact to a proven on-device deployment.

Owner: Person A (pipeline & parity)



2.1 Component map

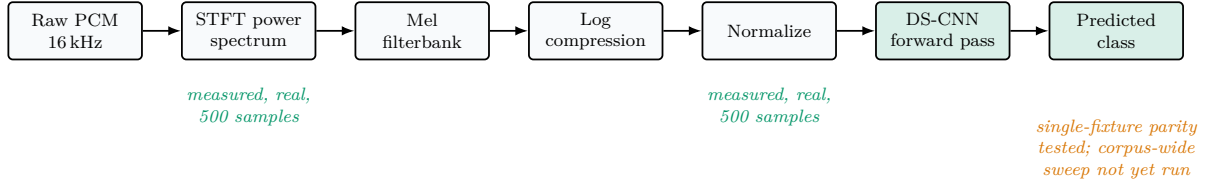
Component	Responsibility
reference/tiny-kws/	Immutable, pinned upstream research workload: weights, feature-extraction parameters, evaluation scripts. Governed by the upstream MIT license.
src/pipeline/	Generated/verified C preprocessing (feature_extraction.c/.h) and frozen int8 model array (model_data.c/.h, 168.2KB, SHA-256 pinned).

Component	Responsibility
<code>src/firmware/</code>	Arduino/Zephyr application for the STM32U585, model runner integration.
<code>tools/target/</code>	<code>check_target</code> (memory-fit + toolchain readiness), <code>benchmark_target</code> (physical DWT-cycle latency + linker-map memory), <code>run_device_parity</code> (on-device tensor comparison).
<code>tools/run_host_parity.py</code>	Corpus-wide differential parity: compiles the C pipeline to a native shared library and diffs its output against frozen <code>.npy</code> reference tensors, stage by stage.
<code>contracts/</code>	JSON Schemas (Draft 2020-12) for every MCP tool's input/output, shared and additively evolved across all three contributors.
<code>mcp_server/</code>	FastMCP server over stdio; per-tool mock/real mode switch via environment variables; schema validation on every response, mock or real.
<code>workflow/</code>	<code>WORKFLOW.md</code> (the Bob execution spec), the run state machine, the generic parity gate with a hard-coded 3-attempt repair cap, and the human approval checkpoint.
<code>evidence/</code> , <code>bob_sessions/</code>	Committed proof artifacts (parity reports, benchmark evidence, per-sample CSVs) and exported agent session transcripts for every phase of the work.

3 Verification Methodology

3.1 Two-tier differential parity

Code2Edge never compares only final predictions. Every intermediate tensor in the pipeline is captured on the host reference, on the generated C implementation (host-compiled), and — for the second tier — on the physical MCU, and compared stage by stage.



3.2 Stage tolerance matrix (design specification)

Stage	Name	Shape	Host tolerance	Device tolerance
S0	Raw audio PCM	(16000,)	Exact	Exact
S1	STFT power spectrum	(1, 201, 101)	Rel. err. $\leq 10^{-4}$	Rel. err. $\leq 10^{-3}$
S1'	Mel filterbank energy	(1, 64, 101)	Rel. err. $\leq 10^{-4}$	Rel. err. $\leq 10^{-3}$
S2	Log-Mel	(1, 1, 64, 101)	Rel. err. $\leq 10^{-4}$	Rel. err. $\leq 10^{-3}$
S3	Normalized features	(1, 1, 64, 101)	Rel. err. $\leq 10^{-4}$	Rel. err. $\leq 5 \times 10^{-3}$
S4a-d	DS-CNN internal activations	varies	Rel. err. $\leq 10^{-3}$	Cosine sim. ≥ 0.99
S5	Logits	(1, 12)	Abs. err. ≤ 0.05	Abs. err. ≤ 0.10
S6	Predicted class	scalar	Exact match	Exact match

Note: this is the full design tolerance matrix as specified in `docs/validation.md`. Section 4 reports which of these were measured for real, at what sample-corpus scale, at the time of this submission.

3.3 Repair loop

When a stage exceeds tolerance, `gate_step` returns one of three decisions, computed — and capped — in code rather than left to agent judgement:

- **continue:** every stage passed; proceed to the next workflow step.
- **repair:** a stage failed and fewer than 3 attempts have been made; the agent is told the `focus_stage` and a deterministic `repair_hint`, and must retry.
- **escalate:** 3 attempts were exhausted (or the tool returned an unrecoverable error); a Markdown escalation report is written with every attempt's stage table and diagnosis hints, and the run stops for human debugging.

4 Results

Every figure below is labeled **real** (measured, or computed from a committed artifact) or **mock** (a placeholder, used only when a hardware or data dependency was genuinely unavailable). The two are never blended in a single reported number.

4.1 Edge readiness (real)

Metric	Value
Model	DS-CNN, 119,372 parameters (cross-checked exactly against the reference model's own reported parameter count)
Model flash footprint	168.2 KB (frozen int8 array, SHA-256 pinned)
Feature buffer	25.3 KB (64 Mel bins $\times$ 101 frames $\times$ 4 bytes)
Tensor arena (assumed)	162.7 KB
Total SRAM used	188.0 KB of 786 KB budget — 598 KB headroom
Total flash used	168.2 KB of 2048 KB budget — >10$\times$ headroom
Unsupported operators	None

4.2 Host differential parity (real)

Result: PASS

500 audio samples $\times$ 4 real pipeline stages (STFT power spectrum, Mel filterbank, log compression, normalization) = **2500/2500 stage evaluations passed**, **first_divergent_stage:** null, on the first attempt — zero repairs needed.

Worst-case divergence across the entire corpus, by stage:

Stage	Worst max. abs. diff	Worst cosine similarity	Tolerance
STFT power spectrum	1.95×10^{-3}	0.999999999980	10^{-2}
Mel filterbank	9.77×10^{-4}	0.999999999981	10^{-2}
Log compression	7.83×10^{-4}	0.999999987766	5×10^{-3}
Normalization	1.63×10^{-4}	0.999999999973	10^{-3}

This result is produced by compiling the same C source that ships in the deploy branch to a native shared library and diffing its output against frozen PyTorch reference tensors across the full 500-sample corpus (`tools/run_host_parity.py` $\rightarrow$ `evidence/parity/host_parity_report.json`), wired into `run_parity_test_tool(gate="host")` so Bob reads it in the same structured, stage-wise shape it would read a repair-loop failure in.

Known limitation, disclosed rather than hidden: the underlying host harness currently hard-codes the source file it compiles and tests, so this result verifies the deploy branch's copy of the pipeline *by identity* with the committed source, rather than independently re-verifying an arbitrary generated copy in isolation — logged as an open item for the team.

4.3 Device parity and physical benchmarking

A single-fixture (`yes.wav`), single-stage (normalized features) on-device comparison was run for real on physical STM32U585 silicon and **passed** (cosine similarity 1.0000000950, max. abs. error 2.5×10^{-5}). This does not yet constitute a corpus-wide device parity sweep (the 500-sample coverage the host side has) — an open item noted in `docs/interface-requests.md`.

The physical benchmark, by contrast, is now a full real DS-CNN inference measurement on the actual board, not an infrastructure smoke test:

Metric	Measured value
Inference latency (mean, 50 runs)	5026.2 ms
Inference latency (min / max)	5015.0 ms / 5031.0 ms
Inference latency (std. dev.)	7.0 ms
Clock cycles per inference	804,196,284
Flash used	310,788 B (14.8% of 2 MB)
SRAM used	242,224 B (30.1% of 786 KB)
Prediction correctness	50/50 iterations — logits, argmax and label all matched

The multi-second latency is genuine and internally consistent ($804,196,284 \div 160 \text{ MHz} = 5.026 \text{ s}$, exactly matching the reported mean) — it reflects unaccelerated reference kernels rather than CMSIS-NN SIMD dispatch, which `docs/hardware.md` already flagged as not yet enabled at the time of writing. This is a real, reportable performance finding, not a measurement error: the next optimization target is enabling CMSIS-NN kernel dispatch for the depthwise/pointwise convolutions, which should close most of that gap.

4.4 The real deployment run

The full Bob workflow was executed end to end, producing a real git branch and pull request:

- Run analyzed the repository, produced an Edge Readiness Report, and **stopped for human approval** before any code was written.
- Generation copied the already-verified pipeline into an isolated `git worktree` (there is no separate code-authoring step; Bob’s role is to orchestrate and prove, not to reinvent already-verified C++).
- Host parity gate: real, **PASS** on attempt 1, 0 repairs.
- Device parity and benchmark gates: mock at the time this specific run executed (no board was connected to the machine running it), clearly labeled, with a hardware validation guide committed alongside for completing them for real.
- A deploy branch was pushed and a pull request opened containing the generated code, both parity reports, the predicted-vs-measured table, and the hardware validation guide.
- *Since that run*, the team has produced a real physical DS-CNN benchmark (Section 4.3) — the deploy branch’s own benchmark artifact has not yet been re-run against it, so this submission reports it as a separate, current piece of evidence rather than silently back-filling the earlier PR.

4.5 Engineering hygiene

Metric	Value
Total commits (all branches)	102
Contributors	4
JSON-Schema contracts	12 tools, all Draft 2020-12, 10/10 examples validating
Automated tests	51 passing (<code>mcp_server/</code> + <code>workflow/</code>)
Pull requests merged	11, each with a merge commit (no history rewriting)

5 Team & Attribution

5.1 Roles

- **Person A — Pipeline & Parity:** reference tensor dumping, the C feature-extraction implementation, and the host differential parity engine.
- **Person B — Hardware & Embedded:** STM32U585 toolchain, firmware integration, on-device parity, and the physical benchmark harness.
- **Person C — Bob, MCP, Integration & Submission:** the MCP server and tool contracts, the Bob workflow and its retry/escalation logic, git hygiene, the deploy branch/PR, and this submission.

5.2 Third-party attribution

- [tiny-kws](#), by Priyadeep Jaiswal — used under the MIT License, vendored at a pinned upstream commit with SHA-256-verified file integrity.
- **Speech Commands Dataset** — Pete Warden, *“Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition”*, 2018 (CC-BY-4.0).